An independent partial check of the three-dimensional Yukawa bcc-to-fcc threshold of Luo and Wei: the threshold reproduced three ways, seventeen finite statements kernel-checked, and what remains open

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-27

Zenodo DOI to be assigned on release (not peer reviewed)

AI-assistance disclosure. This preprint, its Lean code, its Python code and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements for adequacy, and no human referee has checked the hand arguments. **No independent audit of any kind (human or AI) exists at the time of writing**; the producer-is-not-verifier clean recompile is pending, and no statement lock exists.

Credit. The result discussed here is due to Senping Luo and Juncheng Wei, *On exact solutions to three-dimensional Yukawa lattice model and BCC to FCC phase transitions*, arXiv:2610.31456v1 [math.AP], 22 October 2026 [1]. They determine the global minimisers of the Yukawa lattice energy at every screening and prove a single bcc-to-fcc transition at $\kappa_* \approx 1.7179218684699$. The credit for that result belongs to them. This note is a *check* of part of their proof, not a refutation: **every printed number and inequality that we could check holds, and we found no discrepancy.**

Abstract

For unit-density point charges that repel through the screened Coulomb (Yukawa) potential $e^{-\kappa r}/r$, Luo and Wei prove that the body-centred cubic (BCC) lattice has the lowest energy among all three-dimensional lattices for $\kappa < \kappa_*$, the face-centred cubic (FCC) lattice for $\kappa > \kappa_*$, and that $\kappa_* \approx 1.7179218684699$. Their proof leans on finite, computer-assisted inequalities. We checked part of them independently. **We found no discrepancy; every printed number and inequality that we checked holds.** Tier split. [A] (Lean 4.34.1, pinned Mathlib; axioms within {`propext`, `Classical.choice`, `Quot.sound`}; `leanchecker` replay rc 0 on `Basic`, `Shells`, `Chains`, `LemmaA5`, `Lemma61`, run by the producer; no statement lock; no independent recompile yet): *seventeen finite statements*: the FCC shell multiplicities $N_1, \dots, N_{12}$ and the 17 nonzero BCC multiplicities M_ℓ ($\ell \le 48$), proved equal to the count over all of $\mathbb{Z}^3$; Lemma A.5 (A.6), (A.7) and (A.8) as real-number statements over `Real.pi` and `Real.exp`; the constants of the proof of Lemma 6.1; the tail rational of the proof of Lemma A.6; and the rational step of Lemma A.1. [L]: the paper's own statements, taken as stated. [C]: Arb ball arithmetic (Lemma A.6 for $G = 2(E_B - E_F)$ at $\kappa \in \{1.717921868469903, 1.717921868469904\}$ and at neighbouring controls; the Lemma A.5 minimum slacks 7.52×10^{-4} and 7.65×10^{-5}; Lemmas A.1 and A.3) and *three independent reproductions of* κ_* (Ewald sums in Arb at 320 bits; a direct real-space Arb sum; mpmath quadrature of the paper's integral equation (1.4) at 40 digits), agreeing to about 22 digits (IEEE double alone reaches only 11 digits). **Not done:** the assembly of Lemma 6.1 (theta functions as lattice sums, Poisson summation for $D(1) = 0$, $N_n < 30n$ for all n); Lemma A.4

(5 828 rectangles: the partition data is not public and arXiv lists no ancillary files) and Lemma A.2; and Lemma A.6 in Lean (the pinned Mathlib has no `erfc`). Every claim in the text carries a tier tag and a ledger identifier (`YBC-·-nnnn`).

What this is NOT. This concerns the *stated mathematical model* of point charges with screened Coulomb repulsion on a lattice at zero temperature. It makes **no claim about any real material**, plasma, colloid or crystal. It is not a verification of Theorem 1.1 of Luo and Wei, and not a refutation of it.

Markers and ledger. [A]: kernel-checked Lean, axioms within the allowlist, controls caught. [L]: a cited source, taken as stated. [C]: executed code with an argument written down but not machine-checked in Lean (here: interval/ball arithmetic and quadrature). [X]: floating point, supporting nothing by itself. A [A] tag attaches to the Lean statement, never to its physical gloss. Identifiers `YBC-·-nnnn` refer to `ledger/ledger.json` (Section 8).

1 Setting and scope

This is item M3 of a computational programme (Proposal 4, preregistered 2026-10-27 in `results/proposal4/2026-10-27/preregistration.json`; the file was not edited). The preregistered aims were: (M3a) kernel-check Lemma A.5 and Lemma 6.1; (M3b) kernel-check the two signed bounds of Lemma A.6; (M3c) re-run Lemma A.4 (5 828 rectangles) and Lemma A.2 in ball arithmetic. The kill rule said: if rational exp enclosures at the 10^{-20} margin were too slow in the kernel, kernel-check Lemma 6.1 only and report A.6 as Tier C. The outcome is the preregistered *reduced* one, with two differences set out in Section 9: A.6 went to Tier C because Mathlib lacks `erfc`, not because the kernel was too slow, and Lemma 6.1 itself is *not* in Lean, only its finite inputs. Overall status: **PARTIAL**. The prior assigned before any attempt was 0.45 for the headline.

We never write "Luo and Wei is verified". We say which finite statements were checked, at which tier, and which parts were not touched. Had a discrepancy appeared, the protocol was to re-check it a second time and tell the authors before saying anything public.

2 A step-by-step account for non-specialists

Each step ends with the Lean statement that corresponds to it, when there is one. Lean statements are transcribed in ASCII; the sources are in `data/lean/`.

Step 1: the question. [L] Put point charges at the points of a lattice (a regular 3D grid generated by three vectors) with one point per unit volume. If the charges repel as $1/r$ (Coulomb), the lowest-energy lattice is BCC: a cube with a point at each corner and one at the centre. If the repulsion is *screened*, $e^{-\kappa r}/r$ (Yukawa), with screening $\kappa > 0$, strong screening makes the interaction short-ranged, and nearest-neighbour geometry wins: FCC, a cube with points at the corners and face centres, packs the nearest neighbours better. The energy per particle is $E_\kappa(L) = \frac{1}{2}\sum_{v\in L\setminus\{0\}} e^{-\kappa|v|}/|v|$. Luo and Wei prove that among *all* lattices of unit covolume the minimiser is BCC for $\kappa < \kappa_*$ and FCC for $\kappa > \kappa_*$.

Step 2: what a "certified threshold" means. [L] The threshold κ_* is the unique κ with $E_\kappa(L_\mathrm{B}) = E_\kappa(L_\mathrm{F})$. A *certified* enclosure of κ_* means this: evaluate $G(\kappa) = 2(E_\kappa(L_\mathrm{B}) - E_\kappa(L_\mathrm{F}))$ at two rational values $\kappa_1 < \kappa_2$ and show rigorously that $G(\kappa_1) < 0 < G(\kappa_2)$. Because G has a single sign change, $\kappa_* \in (\kappa_1, \kappa_2)$. Lemma A.6 does this with $\kappa_1 = 1717921868469903/10^{15}$ and

$\kappa_2 = 1717921868469904/10^{15}$, claiming $G(\kappa_1) < -10^{-19}$ and $G(\kappa_2) > 10^{-20}$. The numbers are small because $G'(\kappa_*) \approx 1.57 \times 10^{-4}$: one unit in the fifteenth decimal moves G by about 1.6×10^{-19}. [C]

Step 3: shell multiplicities. [L] The FCC lattice L_{F} and its dual BCC lattice $L_{\mathrm{B}} = L_{\mathrm{F}}^*$ have squared lengths $2^{1/3}n$ (with N_n vectors) and $2^{-4/3}\ell$ (with M_ℓ vectors). Concretely $N_n = \#\{m \in \mathbb{Z}^3 : |m|^2 = 2n\}$ and $M_\ell = \#\{m \in \mathbb{Z}^3 : |m|^2 = \ell,\ m_1 \equiv m_2 \equiv m_3 \bmod 2\}$. Everything below uses the printed tables. We recounted them in the kernel, over a finite box, *and* proved that the box count equals the count over all of $\mathbb{Z}^3$:

```
theorem Nc_eq_ncard (n : N) (hn : 2*n < 36) :
    Nc n = Set.ncard {m : Z x Z x Z | normSq m = 2*n}
theorem Mc_eq_ncard (l : N) (hl : l < 49) :
    Mc l = Set.ncard {m : Z x Z x Z | normSq m = l /\ sameParity m}
theorem fcc_multiplicities :
    (List.range 12).map (fun i => Nc (i+1)) = [12,6,24,12,24,8,48,6,36,24,24,24]
theorem bcc_multiplicities :  -- 17 nonzero M_l, l <= 48
    filter (Mc l /= 0) [1..48] = [3,4,8,11,12,16,19,20,24,27,32,35,36,40,43,44,48] /\
    map Mc [3,4,8,...,48] = [8,6,12,24,8,6,24,24,24,32,12,48,30,24,24,24,8]
```

[A] (`YBC-A-0001`, `YBC-A-0002`). The box sizes (radius 5 and 6) are justified by the first lemma: any vector of squared length below $(R+1)^2$ has all coordinates in $[-R, R]$.

Step 4: a finite inequality becomes a statement about real numbers. [L] Lemma A.5 compares two sums of exponentials on a grid of $1000 + 2000$ cells. Here is (A.7): for every integer $0 \le j < 2000$, $\sum_{\ell \le 48} M_\ell e^{-\pi 2^{-4/3}\ell\,(11/10+(j+1)/2000)} - \sum_{n \le 12} N_n e^{-\pi 2^{1/3} n\,(11/10+j/2000)} - 10^{-18} > 10^{-4}$. In Lean, over the reals with `Real.pi`, `Real.exp`, $a_2 = 2^{1/3}$ and $b_2 = 2^{-4/3}$:

```
theorem lemmaA5_A7 (j : N) (hj : j < 2000) :
  sum l in Icc 1 48, (Mc l : R) * exp (-(pi*b2) * l * (11/10 + (j+1)/2000))
  - sum n in Icc 1 12, (Nc n : R) * exp (-(pi*a2) * n * (11/10 + j/2000))
  - 1/10^18 > 1/10000
```

[A] (`YBC-A-0004`); (A.6) is `lemmaA5_A6` for $j < 1000$ with margin $3/1000$ (`YBC-A-0003`), and (A.8) is `8 * exp((pi*a2 - 3*(pi*b2)) * (21/10)) > 25/2` (`lemmaA5_A8`, `YBC-A-0005`).

Step 5: how the kernel checks 3 000 exponential inequalities. [A] The kernel cannot evaluate e^x, so we reduce each cell to rational arithmetic. (i) π is bracketed by Mathlib's `Real.pi_gt_d20` and `pi_lt_d20`; $2^{1/3}$ and $2^{-4/3}$ are bracketed through their cubes. (ii) For a rational z, $e^z = (e^{z/8})^8$ and `Real.exp_bound` gives a rational enclosure from a Taylor polynomial. (iii) The values $e^{-\pi a_2(1+k/10^4)}$ and three similar sequences are supplied as lists of dyadic rationals (denominator 2^{52}) by an *untrusted* script; the kernel checks that each entry follows the previous one within a certified enclosure (`chainLo`, `chainHi`), and the chain is lifted by induction to the real sequence. Because shell sums are monotone in the exponential, one table entry per cell suffices. (iv) Each of the 3 000 cell inequalities is a rational comparison decided by `decide +kernel`, in blocks of 100 cells. An emitter script mirrors every kernel check in exact `Fraction` arithmetic before Lean runs; its slacks, 7.5214×10^{-4} for (A.6) and 7.6459×10^{-5} for (A.7), match the Arb values. The untrusted tables cannot make a false statement pass: a tampered entry fails (Section 6).

Step 6: the proof of Lemma 6.1, as far as it is finite. [L] Lemma 6.1 says $D(t) = \Theta(t, L_{\mathrm{B}}) - \Theta(t, L_{\mathrm{F}})$ satisfies $D(1) = 0$ and $D(t) > 0$ for $t > 1$, where $\Theta(t, L) = \sum_{v \in L} e^{-\pi t |v|^2}$. The printed proof uses Poisson summation (for $D(1) = 0$, since $L_{\mathrm{B}} = L_{\mathrm{F}}^*$), then, for $t \ge 1$, bounds the

shells beyond $n = 13$ and $\ell = 49$ using $N_n < 30n$, $M_\ell < 30\ell$ and a few numerical constants. Those constants are [A] (YBC-A-0006), for example

```
theorem lemma61_exp_fcc : exp (-(pi*a2)) < 1/49
theorem lemma61_exp_bcc : exp (-(pi*b2)) < 3/10
theorem lemma61_tail_fcc (K : N) :
  30 * sum k in range K, (13+k) * (1/49)^(13+k) < 1/10^18
```

The statement for *every partial sum* is what we proved; passing to the infinite series is not written in Lean. The assembly is Section 7.

Step 7: Lemma A.6 and why it is in Ball arithmetic, not Lean. [C] E_κ is not a finite sum. The paper (eqs. 2.3, 2.4) rewrites it by an Ewald split into rapidly convergent sums over L_F and L_B (truncated at $n \le 12$, $\ell \le 48$) whose terms involve an incomplete integral $I(q, c) = \int_1^\infty t^{-1/2} e^{-\pi q t - c/t} dt$, a complementary error function in closed form. The pinned Mathlib has no `erfc`, hence no certified enclosure for I in Lean without building that theory. We therefore verify Lemma A.6 in Arb (Section 5). The rational tail, $60(1/49)^{13}/(1 - 1/49) + 60(3/10)^{49}/(7/10) < 7 \times 10^{-21}$ (exact value 6.5267×10^{-21}), is [A] (`lemmaA6_tail`, YBC-A-0007); so is the rational step $55995/10^6 < 7/125$ of Lemma A.1 (`lemmaA1_rational`).

3 Anatomy of the Lean development

The Lean modules are in `formal_cert/Programmes/YukawaBccFcc/` (copies in `data/lean/`). They import `Mathlib.Tactic` and `Mathlib.Algebra.BigOperators.Fin`; never `import Mathlib`. They were compiled through `scripts/certified_numerics/lean_pinned.py` (Lean 4.34.1, Mathlib `d13f23b7`) and not through `anse/formal/lean_runner.py`, a deviation from the repository's usual route that was inherited from earlier lanes.

Module	Content	compile (s)	leanchecker
`Basic`	box counts and their `ncard` equality; π, $2^{1/3}$, $2^{-4/3}$ brackets; `exp` enclosure; chain lifting; monotone shell polynomials; geometric tail lemma	17.8	rc 0
`Shells`	`decide +kernel` enumerations; `fcc_multiplicities`, `bcc_multiplicities`	657	rc 0
`Chains`	four untrusted dyadic tables (denominator 2^{52}), each step kernel-checked	162	rc 0
`LemmaA5`	3 000 cell checks in 30 blocks of 100; `lemmaA5_A6`, `lemmaA5_A7`	2 009	rc 0
`Lemma61`	(A.8), the Lemma 6.1 constants and tails, the A.6 tail, the A.1 rational step	23	rc 0
`Axioms`, `Statements`	`#print axioms` and `#check` of the exported theorems	15, 12	—
`Controls`, `Reject*`	planted and tampered controls (never imported)	12–32	—

Table 1: The development. Times are wall-clock on a shared core, from the compile logs; `leanchecker` replays were run by the producer.

The tables that the kernel does not trust. The dyadic tables for $e^{-\pi a_2(1+k/10^4)}$, $e^{-\pi b_2(1+k/10^4)}$ and the two grids with step 1/2000 are produced by a script. The kernel never accepts a table as given: each entry must lie between the lower and upper rational enclosures obtained from the previous one, and the first entry and the step constant are checked against certified Taylor enclosures.

A wrong table entry makes the chain check evaluate to `false`, which is what the control in Section 6 shows.

The multiplicities. For reference, the values proved (Step 3) are

n	1	2	3	4	5	6	7	8	9	10	11	12
N_n	12	6	24	12	24	8	48	6	36	24	24	24

ℓ	3	4	8	11	12	16	19	20	24	27	32	35	36	40	43	44	48
M_ℓ	8	6	12	24	8	6	24	24	24	32	12	48	30	24	24	24	8

The sum $N_1 + \cdots + N_{12}$ and these entries are the values printed in the proof of Lemma A.5; we enumerated them independently (`code/m3_shells.py`) rather than reading them from the paper, and the kernel reproduced them.

How to read a Tier A line. `lemmaA5_A7` says: for the specific real numbers named, with the Mathlib constant π and the Mathlib exponential, the displayed inequality holds for every $j < 2000$. It does *not* say that the lemma is used correctly in the paper's proof, that the shell tables encode the right lattices (that is the gloss, which a person must compare with the paper's definitions), or anything about physics. The ledger flags this explicitly (`LEDGER_UNAUDITED_TIER_A`).

4 Results at a glance

Tier	Ledger	Statement	Where
[A]	A-0001/2	$N_1..N_{12}$, 17 nonzero M_ℓ ($\ell \le 48$); box count = count over $\mathbb{Z}^3$	Lean
[A]	A-0003/4/5	Lemma A.5 (A.6) $j < 1000$, (A.7) $j < 2000$, (A.8) over $\mathbb{R}$	Lean
[A]	A-0006	constants of the proof of Lemma 6.1 (8 theorems)	Lean
[A]	A-0007	A.6 tail rational $< 7 \times 10^{-21}$; A.1 rational step	Lean
[A]	A-0008	controls: tampered N7, table entry, cell bound rejected; planted `sorry`/axiom caught	Lean
[L]	L-0001/2	Luo–Wei statements, Sections 2–5, taken as stated	paper
[C]	C-0001	κ_* three ways, 22 digits	Arb, mpmath
[C]	C-0002/3	Lemma A.6 signs (full sums and paper's truncation)	Arb
[C]	C-0004/5	Lemma A.5 re-run; Lemma A.1; Lemma A.3 endpoints	Arb
[C]	C-0007/8	$N_n < 30n$ on $n \le 2100$ only; Lemma 6.1 assembly not formalised	enum., reading
[X]	X-0001/2	IEEE double reaches 11 digits; repository guards fail (other lanes)	float, logs

Seventeen statements are kernel-checked: the `#print axioms` output of each of the following is exactly [`propext`, `Classical.choice`, `Quot.sound`] (`data/logs/lean_axioms.json`): `Nc_eq_ncard`, `Mc_eq_ncard`, `fcc_multiplicities`, `bcc_multiplicities`, `lemmaA5_A6`, `lemmaA5_A7`, `lemmaA5_A8`, `lemma61_exp_fcc`, `lemma61_exp_bcc`, `lemma61_pb_lt`, `lemma61_a2_gt`, `lemma61_exp_fcc_21`, `lemma61_prop41_bracket`, `lemma61_tail_fcc`, `lemma61_tail_bcc`, `lemmaA6_tail`, `lemmaA1_rational`. A scan of the five sources for `skipKernelTC`, `addDecl`, `native_decide`, `implemented_by`, `sorry` and `axiom` has a single hit, a doc comment in `Shells.lean` saying there is no `sorry` and no `native_decide`.

5 The threshold κ_* reproduced three ways, and Lemma A.6

Three independent implementations of $G(\kappa) = 2(E_\kappa(L_\mathrm{B}) - E_\kappa(L_\mathrm{F}))$ (YBC-C-0001, [C]). (E) The Ewald split in Arb (python-flint 0.6.0, 320 bits) with closed-form I and $J = e^{-\pi q - c}/(\pi q + c)$, keeping $n \le 60$, $\ell \le 240$, with a rigorous geometric tail. (D) A direct real-space sum $\sum e^{-\kappa r}/r$ in Arb over about 4 000 shells (integer norms up to 3 800 for FCC and 4 100 for BCC), with a rigorous integral tail from $r_3(k) \le 2(2\sqrt{k}+1)^2$. (Q) The paper's own characterisation (1.4), $\int_1^\infty D(t)\,[t^{-1/2}e^{-c/t} - e^{-ct}]\,dt = 0$, by mpmath quadrature at 40 digits with a secant solver. The shell counts entering (D) and (E) were enumerated here, not copied.

(E) Ewald, Arb	$[1.71792186846990386425087 \pm 1.67 \times 10^{-25}]$
(D) direct sum, Arb	[1.71792186846990386424848, 1.71792186846990386425281] (radii $\le 4.5 \times 10^{-25}$)
(Q) eq. (1.4), mpmath	1.71792186846990386425087416661
(F) IEEE double (information)	1.7179218684625486 [X]
printed (Thm. 6.1; (A.9))	1.71792186846990; $1.717921868469903 < \kappa_* < 1.717921868469904$

(E), (D), (Q) agree to about 22 digits and sit inside the printed enclosure. The double-precision direct sum agrees to only 11 digits because of cancellation at $G'(\kappa_*) \approx 1.566 \times 10^{-4}$; it cannot serve as a 13-digit control, so the float control of the preregistration was replaced by (Q).

Lemma A.6 (YBC-C-0002, [C]). With full Ewald sums, Arb gives (radius about 10^{-49}):

κ ($\times 10^{-15}$ in the last digits)	$G(\kappa)$
1.717921868469902 (negative control)	-2.91987×10^{-19}
1.717921868469903 (Lemma A.6 lower)	$-1.35363 \times 10^{-19} < -10^{-19}$
1.717921868469904 (Lemma A.6 upper)	$+2.12616 \times 10^{-20} > 10^{-20}$
1.717921868469905 (negative control)	$+1.77886 \times 10^{-19}$
1 / 3 (far from κ_*)	-1.0745×10^{-4} / $+1.5771 \times 10^{-4}$

The direct sum (D) agrees within its balls at every κ. Under the paper's own truncation (FCC $n \le 12$, BCC $\ell \le 48$) plus its printed tail 7×10^{-21} (YBC-C-0003), $G_\mathrm{trunc} = -1.35362683 \times 10^{-19}$ and $2.12615782 \times 10^{-20}$, so the slacks are 2.84×10^{-20} (lower side) and 4.26×10^{-21} (upper side); the omitted tail is actually 2.83×10^{-26}, five orders below the printed bound. The identity $G =$ eq. (1.4) integral was also checked at both κ (mpmath: $-1.353626547 \times 10^{-19}$, $+2.126160655 \times 10^{-20}$).

Lemma A.5 in Arb (YBC-C-0004, [C]) and Lemmas A.1, A.3 (YBC-C-0005, [C]). All 1 000 + 2 000 cells hold with no failure; the minimum slacks are 7.52×10^{-4} ((A.6), $j = 999$) and 7.65×10^{-5} ((A.7), $j = 0$), and (A.8) has slack 0.143. Lemma A.1: $P(L_\mathrm{F}, 0) = 0.0558589\ldots < 0.055995 < 7/125 = 0.056 < I(16/25, 0) = 0.0561634\ldots$. Printed counts add up: $\sum n_j = 84\,043$ and $1118 + 193 + 86 + 5828 = 7225$.

Worth a human look, not a discrepancy (YBC-C-0006). In Lemma A.3 (A.4), three exponential bounds hold, but only by about 2×10^{-16}: $e^{\alpha_1} = 0.44 - 2.05 \times 10^{-16} \le 11/25$, $e^{\alpha_2} = 0.147 - 1.37 \times 10^{-16} \le 147/1000$, $e^{\beta} = 4/3 + 1.87 \times 10^{-16} \ge 4/3$. The dyadic endpoints look like double-rounded logarithms, so whether the bounds hold depends on the rounding direction. At 320 bits they hold. We flag this as the tightest margin in the appendix we could check, and a place where a reader implementing the lemma in double precision could be misled; we do not call it an error.

6 Controls

Positive controls (all met). κ_* by three independent implementations (above); the printed digits and the (A.9) enclosure are reproduced; shell counts re-enumerated by brute force up to $k = 4200$ and cross-checked against the r_3 shortcut; the mirror of every Lean check in exact `Fraction` arithmetic reproduces the Arb slacks.

Negative controls (all rejected as intended).

- $\kappa = 1.717921868469902$ and $\ldots 905$ give the expected signs ($G < 0$, $G > 0$).
- **Planted wrong multiplicity** $N_7 = 46$ (the true value is 48), `RejectN7.lean`: fails to compile, rc 1, with a kernel error (*application type mismatch*).
- **Tampered table entry**, `C1[500]` multiplied by 1.01 (`RejectChain.lean`): rc 1. **Tampered cell bound**, the (A.7) margin raised to 1/1000 (`RejectCell.lean`): rc 1. For these two the error text is the elaborator's "`decide` failed, reduction got stuck at the `Decidable` instance", which by itself would not distinguish "false" from "stuck". The decisive evidence is `RejectWitness.lean` (rc 0): the theorems `cellA7bad_0_false : cellA7bad 0 = false`, `bad_chain_false : chainLo s1 C1bad = false`, `N7_is_48 : Nc 7 = 48` and `N7_ne_46` are kernel-checked, so the kernel evaluates the tampered checks to `false` and N_7 to 48 (`YBC-A-0008`, [A]).
- **Planted `sorry` and planted axiom** (`Controls.lean`): the file compiles with rc 0, which a compile exit code alone would accept; `#print axioms` reports `[propext, sorryAx, Classical.choice, Quot.sound]` for `ctl_sorry` and `[propext, Classical.choice, Quot.sound, ctl_cheat]` for `ctl_axiom`. Both are caught by the axiom audit.
- **Tampered rectangle** (Lemma A.4): not applicable, since no partition data exists to tamper with.

7 What remains open

- **Lemma 6.1 is not in Lean** (`YBC-C-0008`, [C]). Its finite inputs are [A]; the assembly needs Θ as a `tsum` over $\mathbb{Z}^3$, three-dimensional Poisson summation for $D(1) = 0$ with $L_{\mathrm{B}} = L_{\mathrm{F}}^*$, the argument on cells for D' and D, and $N_n < 30n$, $M_\ell < 30\ell$ for all n, ℓ. We read the paper's proof and checked $N_n < 30n$ ($n \le 2100$) and $M_\ell < 30\ell$ ($\ell \le 4200$) by enumeration only (`YBC-C-0007`). We did not open a second kernel budget for this; it is a separate, large lift.
- **Lemma A.4 and Lemma A.2** were not re-run ([C] aim M3c). Lemma A.4 is a finite computation on a dyadic partition of 7 225 rectangles, of which 5 828 need the (a)/(b)/(c) alternatives. The paper says the partition data accompany the manuscript, but arXiv lists no ancillary files, and rebuilding Section 5 was outside the budget. This is the largest unchecked part of the computer-assisted proof.
- **Lemma A.6 is not in Lean**: no `erfc` in the pinned Mathlib. A kernel proof would need either that theory or thousands of certified $\sqrt{\cdot}$ and exp enclosures at 10^{-21}.
- The analytic reductions of Sections 2–5, Theorems 5.1 and 6.1, and the uniqueness of κ_* are taken as stated ([L], `YBC-L-0002`).
- An independent clean recompile (producer $\neq$ verifier) of the Lean modules and an audit of statement adequacy are owed.

8 Claim ledger

`ledger/build_ledger.py` files 21 claims: 8 at Tier A (`YBC-A-0001..0008`), 2 at L, 9 at C (`YBC-C-0001..0009`, the last being the headline) and 2 at X. Each has a content-addressed evidence blob in `ledger/evidence/`. The Elenchus gate (`tools/ledger.py ledger.json --evidence-dir evidence`; verbatim in `ledger/gate_output.txt`) exits with code **1**: **0 blocking findings and 8 flags** (`LEDGER_UNAUDITED_TIER_A`, one per Tier A row: no *person* has certified that the Lean statements mean what their glosses say). Exit code 1 is the honest state at writing, not a failure to be cleared by editing. There is no AI audit and no statement lock.

9 Limitations

- **Partial.** See Section 7. We do not claim that Theorem 1.1, 5.1 or 6.1 is verified. Tier A certifies Lean statements about the stated model quantities (unit-covolume shell sums with `Real.pi`, `Real.exp`), nothing more.
- **Deviations from the preregistration.** (1) A.6 is Tier C for a reason other than the kill text gives (no `erfc`). (2) Lemma 6.1 is not kernel-checked, so even the fallback "kernel-check Lemma 6.1 only" was not fully met. (3) The two omitted-tail series are bounded on every partial sum; the monotone limit is not in Lean. (4) The IEEE-double control reaches 11 digits, so the 13-digit float control became mpmath at 40 digits. (5) M3c was blocked, so the tampered-rectangle control is N/A.
- **Compute and process.** The Lean runs were on core 6, which another agent's process also used. Two earlier monolithic compiles of `LemmaA5` were killed with signal 9, unattributed, probably memory pressure from about four 5.7 GB Lean processes of other lanes; the file was then split into four modules with 100-cell blocks. The emitter was re-run once after the oleans were built, to regenerate `Lemma61` only (it is deterministic); `Statements.lean` prints the olean types of the five main theorems and they match the sources. `m3_arb.py` was re-annotated after its first run and re-run with identical key numbers.
- **The leanchecker replay was run by the producer**, so it is not an independent recompile. No statement lock was registered.
- **Repository guards fail, for pre-existing reasons in other lanes' files.** `antigravity_guard.py` rc 1 (103 findings, hallucinated-import findings in other lanes); `test_rigor_guard.py` rc 1 (tests with no assertions in other modules); `pytest tests/` rc 2 (28 collection errors; no test ran). None lies in this lane's files; this lane added no tests. We report the outputs as they came (`data/logs/guard_*.txt`, `pytest_tests.txt`) and do not call the repository gate passed.
- Interpreter and shell lapses of the producer: the Lean wrapper and guards ran under system `python3` without `-I`; a few wait loops were used in background shells. Neither touched git.
- **The paper text is not redistributed.** The 27-page text extraction used for reading is not included here, since we did not establish its licence; the paper is cited.

10 Reproducibility

Repository AutoevolveAI, branch `worktree-openai-math-discovery`; HEAD when this paper was written: `e5f38b289314c730ed4960104c91a2902b88f8ef`. Nothing was pushed or published

by this paper's lane. Copies in `data/`: `code/` (scripts), `lean/` (the twelve Lean sources), `logs/`, `arb_results.json`, `extra_results.json`, `float_results.json`, `shells_k4200.json`, `RESULT.md`, `result.json` (sha256 of all inputs). Lean compile times on core 6: `Basic` 17.8 s, `Shells` 657 s, `Chains` 162 s, `LemmaA5` 2009 s, `Lemma61` 23 s; `leanchecker`: 112, 278, 131, 1601 and 85 s, all rc 0, empty output. Commands (repository root):

```
.venv/bin/python -I code/m3_shells.py ; .venv/bin/python -I code/m3_arb.py     # kappa* (E), (D); Lemma A.5,
      A.6
.venv/bin/python -I code/m3_float.py ; .venv/bin/python -I code/m3_extra.py    # (Q) eq. (1.4); A.1, A.3,
     truncation
.venv/bin/python -I code/emit_lemmaA5.py                                       # proposes Chains/LemmaA5/
     Lemma61 tables
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/YukawaBccFcc/Basic.lean   #
      then Shells, Chains, LemmaA5, Lemma61, Axioms
python -I ledger/build_ledger.py ; python -I ledger/run_gate.py                # gate: exit 1 = 0 blocking,
     8 flags
```

sha256 of the Lean sources (`data/lean/`) are in `ledger/evidence/` and `data/result.json`. Toolchain: Lean 4.34.1, Mathlib `d13f23b7`, python-flint 0.6.0, mpmath.

References

[1] S. Luo, J. Wei, *On exact solutions to three-dimensional Yukawa lattice model and BCC to FCC phase transitions*, arXiv:2610.31456v1 [math.AP], 22 Oct 2026. The source of every statement checked here ([L]).

[2] X. Callens, *No connected graph with at most ten vertices is an unweighted death-Birth amplifier of weak selection* (papers/p4_amplifier_census), own preprint in this programme, format and ledger model; not peer reviewed.

[3] X. Callens, *Triangular lattice minimises the planar Yukawa energy*, doi:10.5281/zenodo.23646105 (2026), own preprint; the planar (two-dimensional) Yukawa lattice setting, with the same tier conventions. Not peer reviewed. The present note concerns the three-dimensional problem and does not extend that one.

[4] The mathlib Community, *The Lean mathematical library*, arXiv:1910.09336 (2019); cited for the library only.

[5] F. Johansson, *Arb: efficient arbitrary-precision midpoint-radius interval arithmetic*, IEEE Trans. Comput. 66 (2017) 1281–1292; used through python-flint 0.6.0.